CE 222 – *Computer Organization and Assembly Language (COAL)*
Lecture 20

Dr. Asad Mahmood

March 03, 2026

Lecture Outline

- Announcements (Assignment and Quiz 3)
- Outline of Previous Lecture:
  - **Chapter 3 – Arithmetic for Computers**
    - 3.5 Floating Point

- Outline to Today's Lecture:
  - **Chapter 3 – Arithmetic for Computers**
    - 3.5 Floating Point
    - 3.6 Parallelism and Computer Arithmetic: Subword Parallelism

Chapter 3

Arithmetic for Computers

Chapter Outline

Chapter 3 - Arithmetic for Computers

- 3.1 Introduction
- 3.2 Addition and Subtraction
- 3.3 Multiplication
- 3.4 Division
- 3.5 Floating Point
- 3.6 Parallelism and Computer Arithmetic: Subword Parallelism
- 3.7 Real Stuff: Streaming SIMD Extensions in x86
- 3.8 Going Faster: Subword Parallelism and Matrix Multiply
- 3.9 Fallacies and Pitfalls
- 3.10 Concluding Remarks
- 3.11 Historical Perspective and Further Reading

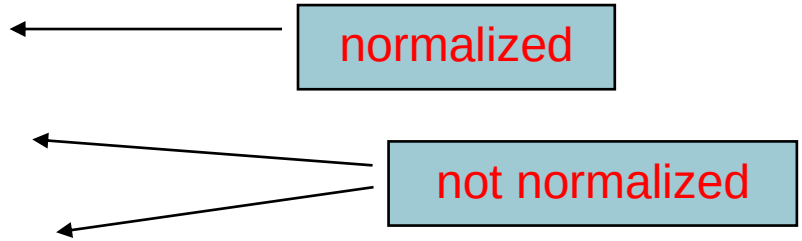
Floating Point

“Speed gets you nowhere if you’re headed the wrong way”

American proverb

Floating Point

- Representation for non-integral numbers
  - Including very small and very large numbers
- Like scientific notation
  - -2.34×10^{56}
  - $+0.002 \times 10^{-4}$
  - $+987.02 \times 10^{9}$



- In binary
  - $\pm 1.xxxxxxx_2 \times 2^{yyyy}$
- Types `float` and `double` in C

Floating Point Representation

- Tradeoff between precision (fraction part) and range (exponent part)
  -> Good design demands good compromise!
- Different levels of range and precision
  - Half precision (16 bits)
  - Single Precision (32 bits) – `float`
    - Range = 2×10^{-38} to 2×10^{38}
    - Extraordinary but not infinity -> overflow still possible!
    - And underflow!

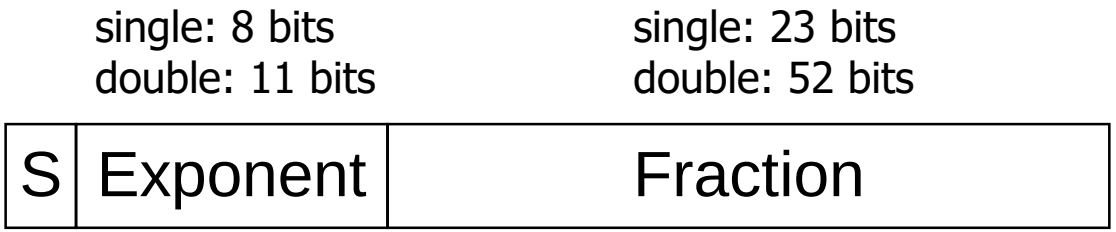
Floating Point Representation

- Different levels of precision
  - Double Precision (64 bits) – double
    - Range = 2×10^{-308} to 2×10^{308}
    - Primary advantage is not the increased range but rather the enhanced precision!

Floating Point Standard

- Defined by IEEE Std 754-1985
- Developed in response to divergence of representations
  - Portability issues for scientific code
- Now almost universally adopted
- Two representations
  - Single precision (32-bit)
  - Double precision (64-bit)

IEEE Floating-Point Format



- Equivalent decimal number
- Normalize significand:
- $1.0 \leq |\text{significand}| < 2.0$
- Exponent: biased representation: actual exponent + Bias
  - Ensures exponent is unsigned
  - Single: Bias = 127; Double: Bias = 1203

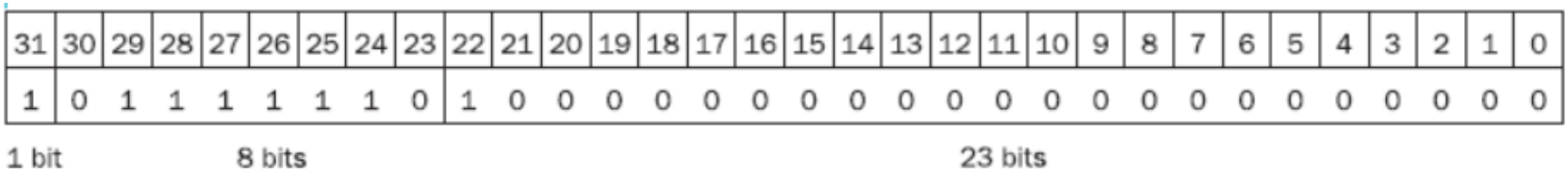
IEEE Floating-Point Format

Single precision		Double precision		Object represented
Exponent	Fraction	Exponent	Fraction	
0	0	0	0	0
0	Nonzero	0	Nonzero	± denormalized number
1–254	Anything	1–2046	Anything	± floating-point number
255	0	2047	0	± infinity
255	Nonzero	2047	Nonzero	NaN (Not a Number)

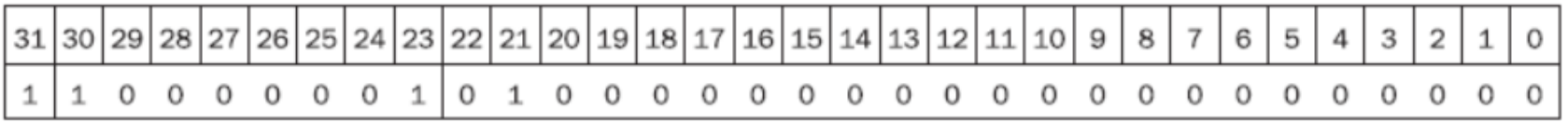
- Exponents 00000000 and 11111111 reserved
- Denormalized numbers are very small nos. between the smallest normalized/normal no. and 0.
- Infinity vs Not-a-number (NaN)
- Smallest value ?
- Largest value ?

Floating-Point Example

- Example 1: Represent –0.75 in single precision format



- Example 2: What decimal number is represented by the following single precision number?



Floating-Point Precision

- Precision = No. of significant digits accurately represented
- For floating points
  - Single Precision
    - 23 fraction bits + 1 = 24 bits can be accurately represented -> 24 bit binary precision
    - Equivalent decimal precision ?
  - Double Precision
    - Binary precision ?
    - Equivalent decimal precision ?

Floating-Point Addition

- How would we add decimal nos. in scientific notation?

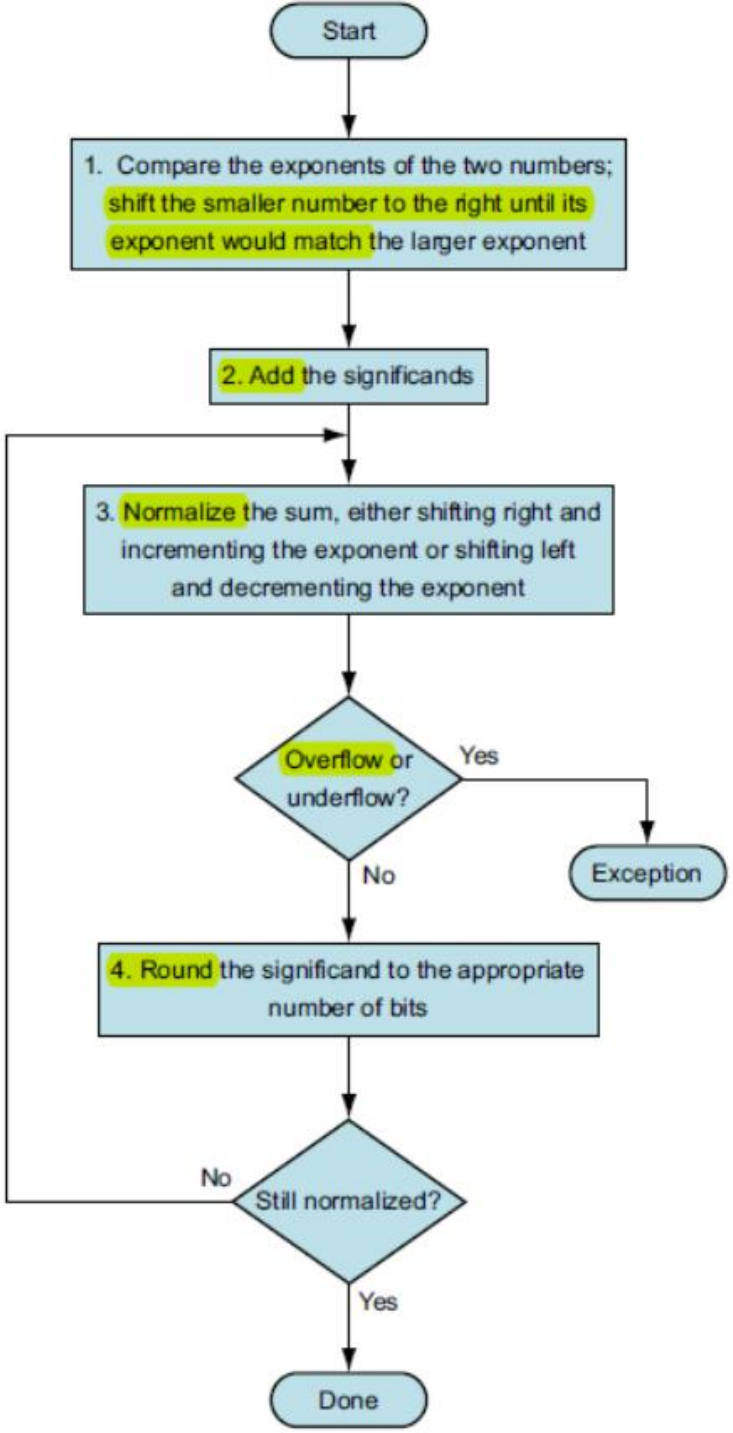
- Process followed:
  1. Align decimal points
     - Shift number with smaller exponent
  2. Add significands
  3. Normalize result & check for over/underflow
  4. Round and renormalize if necessary

Floating-Point Addition

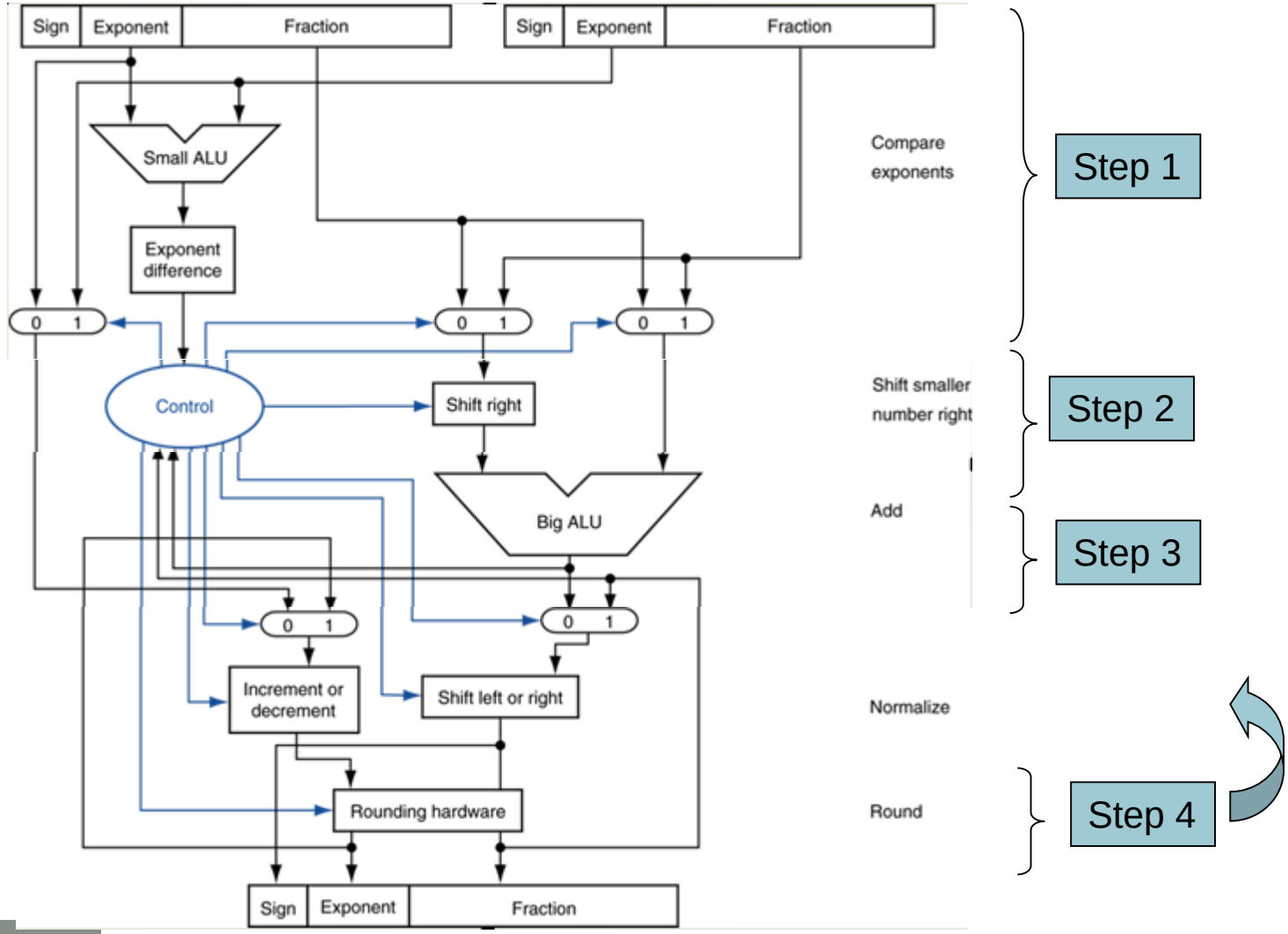
- Same process to follow for binary floating point addition:
  - Example: $(1.000_2 \times 2^{-1}) + (-1.100_2 \times 2^{-2})$

1. Align binary points
2. Add significands
3. Normalize result & check for over/underflow
4. Round and renormalize if necessary

Floating-Point Addition



FP Adder Hardware

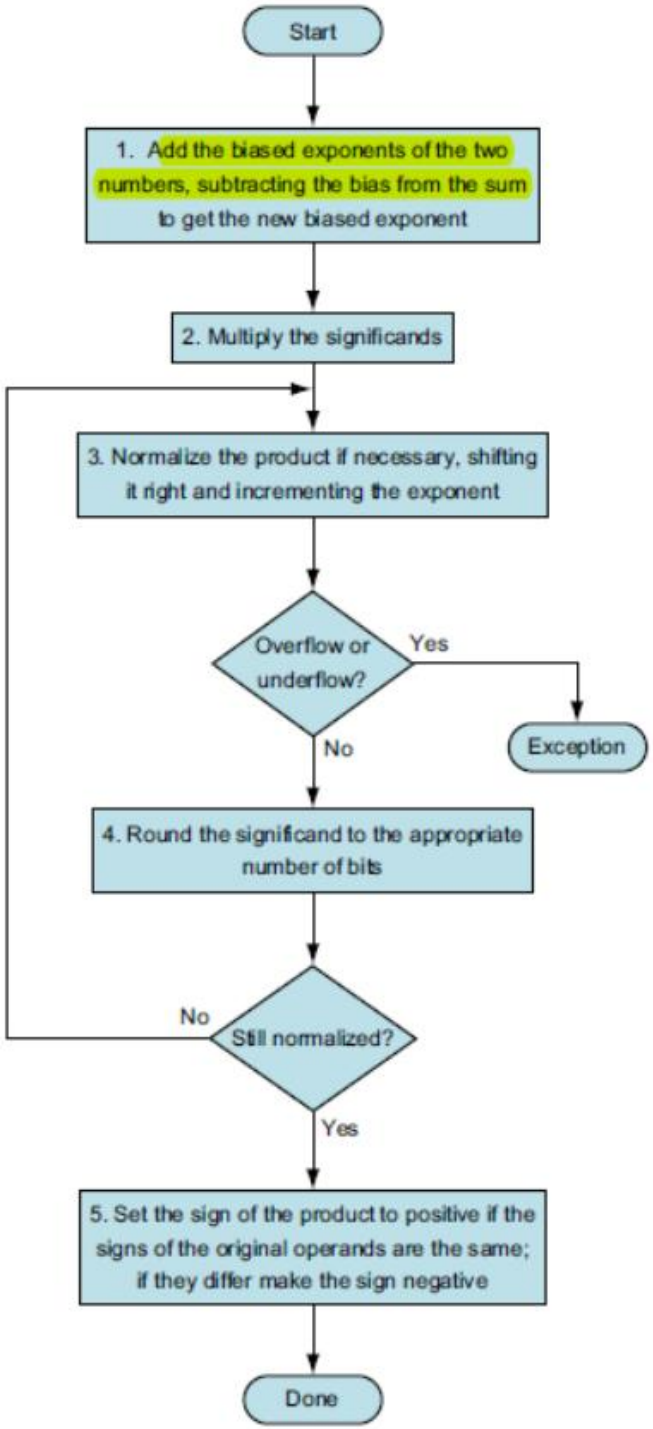


Floating-Point Multiplication

- How would we *multiply* decimal nos. in scientific notation?

- Process followed:
  1. Add exponents
  2. Multiply significands
  3. Normalize result & check for over/underflow
  4. Round and renormalize if necessary
  5. Determine sign of result from sign of operands

Floating-Point Multiplication



Floating-Point Multiplication

- Same process followed for multiplication of floating-point binary numbers
- Example: $(1.000_2 \times 2^{-1}) \times (-1.110_2 \times 2^{-2})$
- Process followed:
  1. Add exponents
  2. Multiply significands
  3. Normalize result & check for over/underflow
  4. Round and renormalize if necessary
  5. Determine sign of result from sign of operands

FP Arithmetic Hardware

- FP multiplier is of similar complexity to FP adder
  - But uses a multiplier for significands instead of an adder
- FP arithmetic hardware usually does
  - Addition, subtraction, multiplication, division, reciprocal, square-root
  - FP $\leftrightarrow$ integer conversion
- Operations usually takes several cycles
  - Can be pipelined

FP Instructions in RISC-V

- Separate FP registers: f0, …, f31
  - double-precision
  - single-precision values stored in the lower 32 bits
- FP instructions operate only on FP registers
  - Programs generally don’t do integer ops on FP data, or vice versa
- Benefit of dedicated FP registers?
- FP load and store instructions
  - `flw, fld`
  - `fsw, fsd`

FP Instructions in RISC-V

- Single-precision arithmetic
  - `fadd.s`, `fsub.s`, `fmul.s`, `fdiv.s`, `fsqrt.s`
    - e.g., `fadds.s f2, f4, f6`
- Double-precision arithmetic
  - `fadd.d`, `fsub.d`, `fmul.d`, `fdiv.d`, `fsqrt.d`
    - e.g., `fadd.d f2, f4, f6`
- Single- and double-precision comparison
  - `feq.s`, `flt.s`, `fle.s`
  - `feq.d`, `flt.d`, `fle.d`
  - Result is 0 or 1 in integer destination register
    - Use beq, bne to branch on comparison result

FP Example: °F to °C

- C code:

```
float f2c (float fahr) {
  return ((5.0/9.0)*(fahr - 32.0));
}
```

  - fahr in f10, result in f10, literals in global memory space

- Compiled RISC-V code:

```
f2c:
  flw    f0,const5(x3)  // f0 = 5.0f
  flw    f1,const9(x3)  // f1 = 9.0f
  fdiv.s f0, f0, f1  // f0 = 5.0f / 9.0f
  flw    f1,const32(x3) // f1 = 32.0f
  fsub.s f10,f10,f1  // f10 = fahr - 32.0
  fmul.s f10,f0,f10  // f10 = (5.0f/9.0f) * (fahr-32.0f)
  jalr   x0,0(x1)    // return
```

FP Example: Array Multiplication

- C = C + A × B
  - All 32 × 32 matrices, 64-bit double-precision elements

- C code:

```
void mm (double c[][],
         double a[][], double b[][]) {
  size_t i, j, k;
  for (i = 0; i < 32; i = i + 1)
    for (j = 0; j < 32; j = j + 1)
      for (k = 0; k < 32; k = k + 1)
        c[i][j] = c[i][j]
                + a[i][k] * b[k][j];
}
```

  - Addresses of c, a, b in x10, x11, x12, and i, j, k in x5, x6, x7

FP Example: Array Multiplication

- RISC-V code:

```
mm:...
        li    x28,32        // x28 = 32 (row size/loop end)
        li    x5,0          // i = 0; initialize 1st for loop
L1:     li    x6,0          // j = 0; initialize 2nd for loop
L2:     li    x7,0          // k = 0; initialize 3rd for loop
        slli  x30,x5,5      // x30 = i * 2**5 (size of row of c)
        add   x30,x30,x6    // x30 = i * size(row) + j
        slli  x30,x30,3     // x30 = byte offset of [i][j]
        add   x30,x10,x30   // x30 = byte address of c[i][j]
        fld   f0,0(x30)     // f0 = c[i][j]
L3:     slli  x29,x7,5      // x29 = k * 2**5 (size of row of b)
        add   x29,x29,x6    // x29 = k * size(row) + j
        slli  x29,x29,3     // x29 = byte offset of [k][j]
        add   x29,x12,x29   // x29 = byte address of b[k][j]
        fld   f1,0(x29)     // f1 = b[k][j]
```

FP Example: Array Multiplication

```
…
    slli   x29,x5,5     // x29 = i * 2**5 (size of row of a)
    add    x29,x29,x7   // x29 = i * size(row) + k
    slli   x29,x29,3    // x29 = byte offset of [i][k]
    add    x29,x11,x29  // x29 = byte address of a[i][k]
    fld    f2,0(x29)    // f2 = a[i][k]
    fmul.d f1, f2, f1   // f1 = a[i][k] * b[k][j]
    fadd.d f0, f0, f1   // f0 = c[i][j] + a[i][k] * b[k][j]
    addi   x7,x7,1      // k = k + 1
    bltu   x7,x28,L3    // if (k < 32) go to L3
    fsd    f0,0(x30)    // c[i][j] = f0
    addi   x6,x6,1      // j = j + 1
    bltu   x6,x28,L2    // if (j < 32) go to L2
    addi   x5,x5,1      // i = i + 1
    bltu   x5,x28,L1    // if (i < 32) go to L1
```

Accurate Arithmetic

- IEEE Std 754 specifies additional rounding control
  - Extra bits of precision (guard, round, sticky) for intermediate results before final rounding off
  - Choice of rounding modes
  - Allows programmer to fine-tune numerical behavior of a computation
- Example

Chapter Outline

Chapter 3 - Arithmetic for Computers

- 3.1 Introduction
- 3.2 Addition and Subtraction
- 3.3 Multiplication
- 3.4 Division
- 3.5 Floating Point
- 3.6 Parallelism and Computer Arithmetic: Subword Parallelism
- 3.7 Real Stuff: Streaming SIMD Extensions in x86
- 3.8 Going Faster: Subword Parallelism and Matrix Multiply
- 3.9 Fallacies and Pitfalls
- 3.10 Concluding Remarks
- 3.11 Historical Perspective and Further Reading